

Chapter 2 Database Environment

Chapter Objectives

In this chapter you will learn:

- The purpose and origin of the three-level database architecture.
 - The contents of the external, conceptual, and internal levels.
 - The purpose of the external/conceptual and conceptual/internal mappings.
 - The meaning of logical and physical data independence.
 - The distinction between a Data Definition Language (DDL) and a Data Manipulation Language (DML).
- A classification of data models.
- The purpose and importance of conceptual modeling.
- The typical functions and services that a DBMS should provide.
- The function and importance of the system catalog.

A fundamental goal of any database system is to present users with an abstract view of their data, concealing the details of how that data is physically stored and manipulated. The starting point for database design must therefore be an abstract, implementation-independent description of the organization's information requirements. Throughout this book, the term "organization" is used loosely to mean the whole organization or a relevant part of it. In the DreamHome case study, for instance, we might be interested in modeling:

- Real-world entities such as Staff, PropertyForRent, PrivateOwner, and Client.
 - Attributes that describe properties of each entity — for example, each Staff entry has a name, a position, and a salary.
 - Relationships between these entities — for example, Staff Manages PropertyForRent.

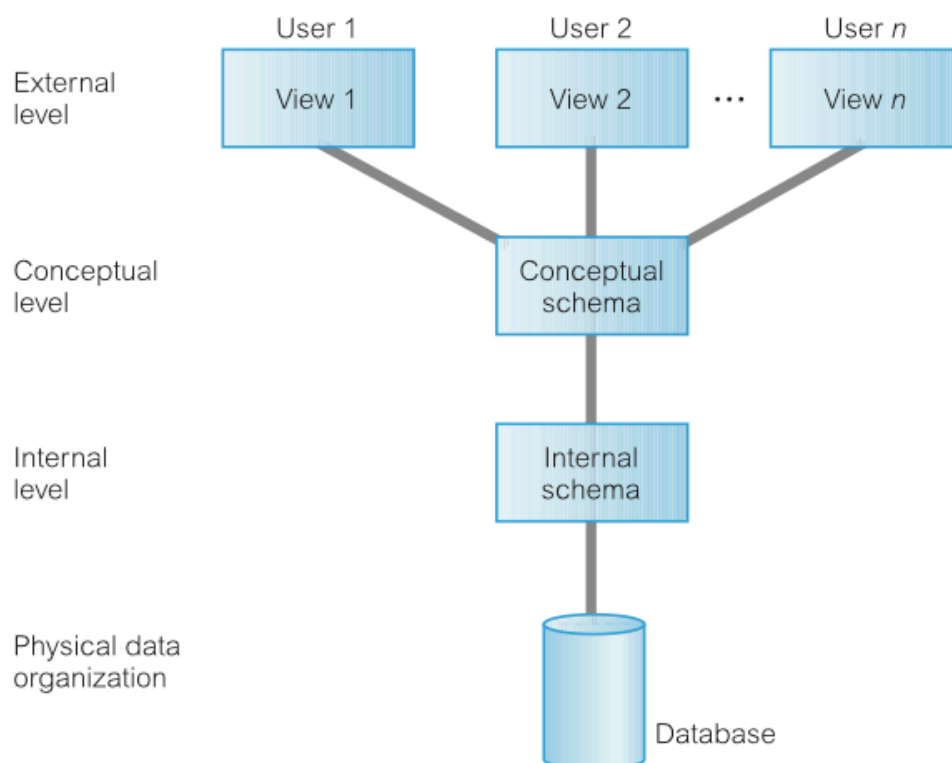
Because a database is a shared resource, different users will typically need different views of the same data. To satisfy this requirement, the architecture of most commercial DBMSs today is grounded to some degree in the so-called ANSI-SPARC architecture. This chapter examines various architectural and functional characteristics of DBMSs.

2.1 The Three-Level ANSI-SPARC Architecture

An early proposal for standard database terminology and a general database architecture was produced in 1971 by the Data Base Task Group (DBTG), which was appointed by CODASYL. The DBTG recognized the need for a two-level approach consisting of a system-wide view called the schema and user-specific views called subschemas. A similar — and more influential — proposal emerged from the American National Standards Institute (ANSI) Standards Planning and Requirements Committee (SPARC), or ANSI/X3/SPARC, in 1975. The ANSI-SPARC architecture extended the DBTG approach to three levels and

introduced the concept of a system catalog. These proposals built on earlier work published by the IBM user organizations Guide and Share, which had emphasized the need for an implementation-independent layer to insulate programs from the specifics of underlying data storage. Although the ANSI-SPARC model was never formally adopted as a standard, it continues to provide a valuable conceptual foundation for understanding DBMS functionality.

The essential contribution of these proposals is the identification of three levels of abstraction — three distinct levels at which data can be described. Together these form the three-level architecture, comprising an external, a conceptual, and an internal level, as depicted in Figure 2.1.



The external level represents the way users perceive the data. The internal level represents the way the DBMS and the operating system store and manage the data physically, using the data structures and file organizations described in Appendix F. The conceptual level sits between these two and provides both the mapping and the desired independence between them.

The overarching goal of the three-level architecture is to separate each user's view of the database from the way the database is physically implemented. Several motivations drive this separation:

- Each user should be able to access the same underlying data while seeing a customized view of it. A user should be able to modify their view without affecting other users.
- Users should be insulated from physical storage details such as indexing and hashing strategies. Their interaction with the database should be independent of storage considerations.
- The DBA should be able to modify physical storage structures without disturbing users'

views.

- Changes to physical storage media or devices should not affect the internal logical structure of the database.
 - The DBA should be able to modify the conceptual structure of the database without affecting all users.
-

2.1.1 External Level

External level: The users' view of the database. This level describes that part of the database that is relevant to each user.

The external level consists of multiple distinct external views of the database. Each user perceives the "real world" represented in a form that is natural and familiar to them. An external view includes only those entities, attributes, and relationships that are relevant to a particular user; anything outside that user's scope of interest, though potentially present in the database, remains invisible to them.

Different views may represent the same data in different forms. One user might view dates as (day, month, year) while another sees them as (year, month, day). Views may also include derived or calculated data — values that are not physically stored but are computed on demand. In the DreamHome case study, for example, users might want to see a staff member's current age. Since age changes daily, it would not be stored directly; instead, the member's date of birth would be stored and the DBMS would calculate age whenever it is needed. Views may even aggregate data drawn from multiple entities.

2.1.2 Conceptual Level

Conceptual level: The community view of the database. This level describes what data is stored in the database and the relationships among the data.

The middle tier in the three-level architecture is the conceptual level. It contains the complete logical structure of the database as perceived by the DBA — a full representation of the organization's data requirements that is entirely independent of storage considerations. The conceptual level captures:

- All entities, their attributes, and the relationships between them.
- Constraints on the data.
- Semantic information about the data.
- Security and integrity information.

The conceptual level must underpin all external views: any data a user can see must be present in, or derivable from, the conceptual level. At the same time, this level must remain completely free of any storage-dependent details. A description of an entity, for instance, should specify data types and field lengths but nothing about how many bytes a field actually occupies in storage.

2.1.3 Internal Level

Internal level: The physical representation of the database on the computer. This level describes how the data is stored in the database.

The internal level governs the physical implementation of the database with the goal of achieving optimal runtime performance and efficient use of storage. It specifies the data structures and file organizations used to store data on storage devices, and it interfaces with the operating system's access methods to place data on storage media, maintain indexes, and retrieve records. The internal level addresses:

- Allocation of storage space for data and indexes.
- Record descriptions for physical storage, including the stored sizes of data items.
- Record placement strategies.
- Data compression and encryption techniques.

Below the internal level lies a physical level managed by the operating system, acting under the direction of the DBMS. The boundary between DBMS responsibility and operating system responsibility at this physical layer is not sharply defined and varies from system to system. Some DBMSs exploit a wide range of operating system access methods; others use only the most basic ones and implement their own file organization strategies. At the purely physical level, only the operating system has full knowledge of matters such as how sequencing is implemented or whether the fields of internal records are stored as contiguous bytes on disk.

2.1.4 Schemas, Mappings, and Instances

The complete description of the database is called the database schema. Three types of schema correspond to the three levels of the ANSI-SPARC architecture.

At the highest level, there are multiple external schemas (also called subschemas), one for each user's view. At the conceptual level, there is a single conceptual schema capturing all entities, attributes, relationships, and integrity constraints. At the lowest level, there is a single internal schema — a complete physical description of the database, specifying stored record formats, representation methods, data fields, indexes, and storage structures. Every database has exactly one conceptual schema and one internal schema.

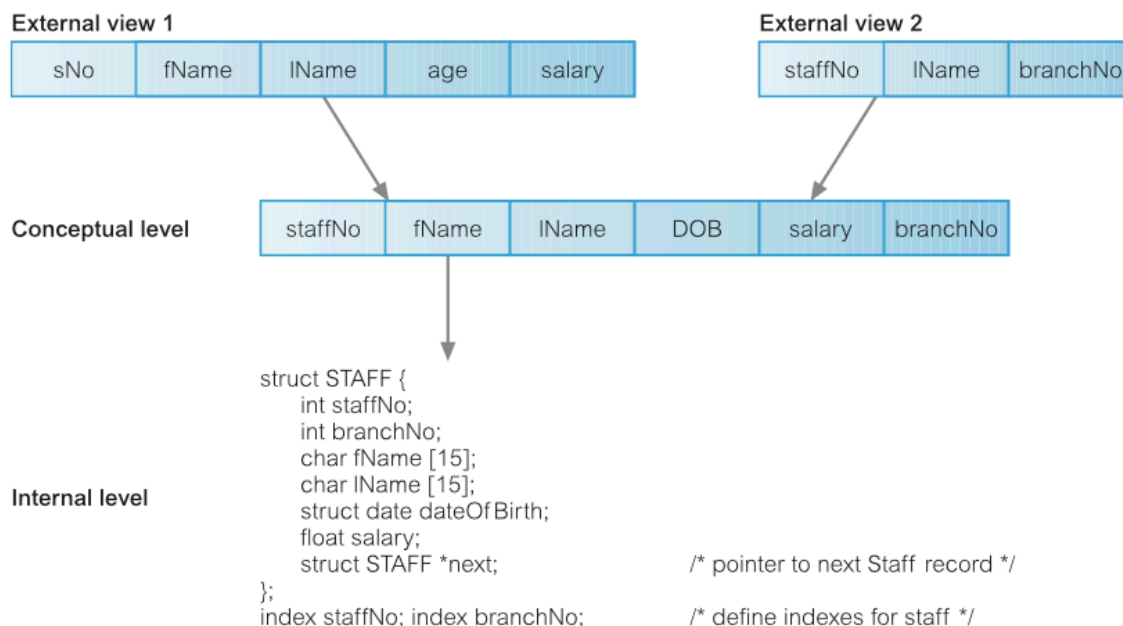
The DBMS is responsible for managing the mappings between these three schema types and for verifying their mutual consistency — specifically, confirming that every external schema is derivable from the conceptual schema, and using the conceptual schema as the bridge when mapping between external and internal schemas.

The conceptual/internal mapping links the conceptual schema to the internal schema. This mapping enables the DBMS to locate in physical storage the actual record or combination of records that constitutes a logical conceptual record, and to enforce any applicable

constraints. It also handles differences in entity names, attribute names, attribute ordering, and data types between the two levels.

The external/conceptual mapping links each external schema to the conceptual schema. This mapping enables the DBMS to translate names used in a user's view to the corresponding elements in the conceptual schema.

Figure 2.2 illustrates these relationships concretely. Two distinct external views of staff details exist: the first comprises staff number (sNo), first name (fName), last name (lName), age, and salary; the second contains staff number (staffNo), last name (lName), and branch number (branchNo). These views are merged at the conceptual level, where the key difference is that the age field is replaced by a date of birth field (DOB). The DBMS maintains the external/conceptual mapping — for instance, mapping the sNo field in the first external view to the staffNo field in the conceptual record. The conceptual level is then mapped to the internal level, which contains a physical description of the record structure in a high-level language. At this level, a pointer named next links staff records together into a chain. Note also that the field ordering at the internal level differs from that at the conceptual level — again, the DBMS maintains the conceptual/internal mapping to resolve such differences.



A crucial distinction must be drawn between the description of the database and the database itself. The database schema — the description — is established during the design process and rarely changes. The actual data stored in the database, however, may change frequently: every time a new staff member or property is added, for instance. The data in the database at any given moment is called a database instance. Many instances can correspond to the same schema over time. The schema is sometimes called the intension of the database; an instance is called an extension, or state, of the database.

2.1.5 Data Independence

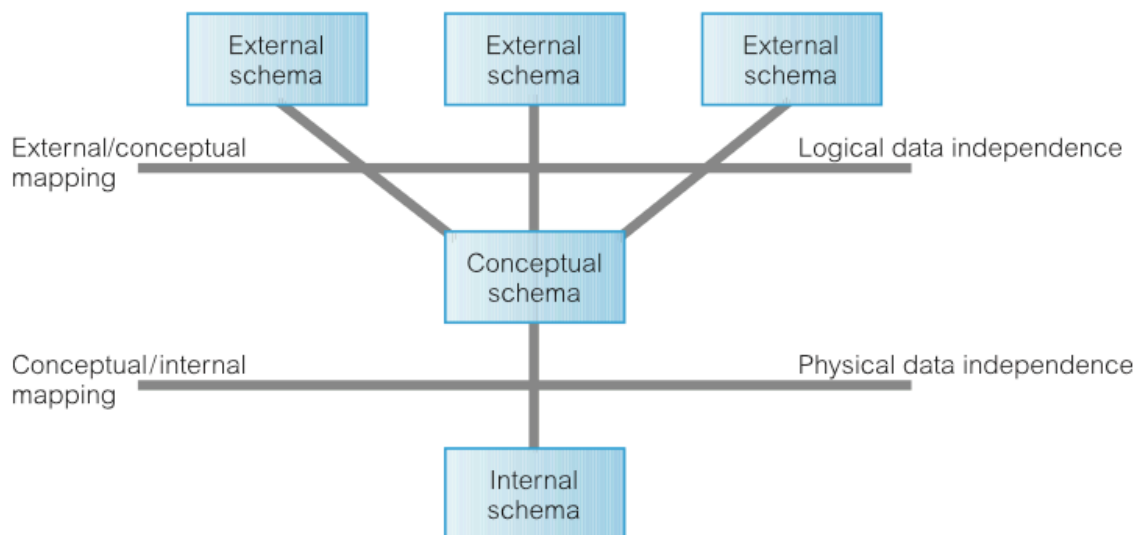
A central objective of the three-level architecture is data independence — ensuring that changes at lower levels do not force changes at higher levels. Two forms of data independence are distinguished.

Logical data independence: The immunity of the external schemas to changes in the conceptual schema.

Changes at the conceptual level — such as adding or removing entities, attributes, or relationships — should not require modifications to existing external schemas or rewrites of application programs. Users affected by a change need to be aware of it, but the important point is that users not affected by the change should not notice anything different.

Physical data independence: The immunity of the conceptual schema to changes in the internal schema.

Changes at the internal level — such as adopting different file organizations, switching to different storage devices, or modifying indexing strategies — should not require any modification to the conceptual or external schemas. The only effect users might notice is a change in system performance. In fact, a decline in performance is the most common reason for making internal schema changes. Figure 2.3 shows where each type of data independence occurs within the three-level architecture.



The two-stage mapping inherent in the ANSI-SPARC architecture introduces some processing overhead but in return provides a high degree of data independence. For efficiency, the architecture permits direct mapping from external schemas to the internal schema, bypassing the conceptual level. This shortcut improves mapping performance but at the cost of data independence: any change to the internal schema may then also require changes to external schemas and their dependent application programs.

2.2 Database Languages

A data sublanguage consists of two parts: a Data Definition Language (DDL) and a Data Manipulation Language (DML). The DDL is used to specify the database schema; the DML is used to read and update the database. These are called sublanguages because they do not provide all the constructs needed for general-purpose computing — conditional statements, loops, and similar constructs are supplied by the host high-level programming language in which the sublanguage is embedded. Many DBMSs support embedding a data sublanguage within a host language such as COBOL, Fortran, Pascal, Ada, C, C++, C#, Java, or Visual Basic. To process an embedded file, the sublanguage commands are first extracted from the host-language program and replaced by function calls. The preprocessed file is then compiled, placed in an object module, linked with a DBMS-specific library containing the substituted functions, and executed as needed. Most data sublanguages also provide non-embedded interactive modes, allowing commands to be entered directly from a terminal.

2.2.1 The Data Definition Language (DDL)

DDL: A language that allows the DBA or user to describe and name the entities, attributes, and relationships required for the application, together with any associated integrity and security constraints.

The database schema is specified using a Data Definition Language. The DDL is used to define a schema or to modify an existing one; it cannot be used to manipulate data.

Compiling DDL statements produces a set of tables stored in special files collectively called the system catalog. The system catalog holds metadata — data that describes the objects in the database and facilitates their access and manipulation. This metadata includes definitions of records, data items, and other objects relevant to users or required by the DBMS. The DBMS consults the system catalog before accessing actual data in the database. The terms data dictionary and data directory are also used to describe the system catalog, though "data dictionary" typically refers to a more general-purpose software system than a DBMS catalog. The system catalog is discussed further in Section 2.4.

In principle, distinct DDLs could exist for each of the three schema levels — one for external schemas, one for the conceptual schema, and one for the internal schema. In practice, most systems provide a single comprehensive DDL that covers at least the external and conceptual schemas.

2.2.2 The Data Manipulation Language (DML)

DML: A language that provides a set of operations to support the basic data manipulation operations on the data held in the database.

Data manipulation operations typically include:

- Insertion of new data into the database.
- Modification of data already stored in the database.
- Retrieval of data from the database.
- Deletion of data from the database.

One of the primary functions of the DBMS is to support a DML through which users can express these operations. Data manipulation applies at all three levels — external, conceptual, and internal — though at the internal level the focus is on complex low-level procedures that enable efficient data access, while at higher levels the emphasis is on ease of use and productive interaction with the system.

The data retrieval portion of a DML is called a query language — a high-level, special-purpose language used to express diverse requests for data retrieval. The term "query" is technically reserved for a retrieval statement expressed in a query language. The terms "query language" and "DML" are frequently used interchangeably, though this conflation is technically imprecise.

DMLs are distinguished by their underlying retrieval mechanisms and fall into two broad categories: procedural and nonprocedural. The key difference is that a procedural DML specifies both what data is needed and how to retrieve it, while a nonprocedural DML specifies only what data is needed, leaving the DBMS to determine how to obtain it. Procedural languages typically operate on one record at a time; nonprocedural languages typically operate on entire sets of records.

Procedural DML: A language that allows the user to tell the system what data is needed and exactly how to retrieve it.

With a procedural DML, the user or programmer specifies the required data and the procedure for obtaining it, invoking explicit data-access operations in sequence. Retrieval is record-by-record: one record is retrieved and processed, the result determines the next retrieval, and so on until the complete dataset has been assembled. Procedural DMLs are typically embedded in host programming languages that provide iteration and navigation constructs. Network and hierarchical DMLs are usually procedural.

Nonprocedural DML: A language that allows the user to state what data is needed rather than how it is to be retrieved.

With a nonprocedural DML, the required data is specified in a single retrieval or update statement. The DBMS translates that statement into the necessary access procedures, freeing the user from any knowledge of internal data structures or algorithms. This provides a substantial degree of data independence. Nonprocedural languages are also called declarative languages. Relational DBMSs typically provide SQL or QBE (Query-By-Example)

as their nonprocedural DML. Nonprocedural DMLs are generally easier to learn and use than procedural ones, because the DBMS assumes responsibility for the implementation details. SQL is examined comprehensively in Chapters 6–9 and Appendix I; QBE is covered in Appendix M.

2.2.3 Fourth-Generation Languages (4GLs)

There is no universally agreed definition of what constitutes a fourth-generation language; in essence, it is a highly concise programming notation. An operation that requires hundreds of lines of code in a third-generation language (3GL) such as COBOL can typically be expressed in far fewer lines in a 4GL.

Where a 3GL is procedural — specifying step by step how a task is to be carried out — a 4GL is nonprocedural: the user defines what is to be accomplished, not how. A 4GL relies heavily on high-level built-in components known as fourth-generation tools. Rather than defining the steps a program must follow, the user supplies parameters to these tools, which then generate an application program. 4GLs are claimed to improve developer productivity by an order of magnitude, at the cost of some constraints on the types of problem they can address. The fourth-generation language category encompasses:

- Presentation languages, including query languages and report generators.
- Specialty languages, such as spreadsheets and database languages.
- Application generators that support the definition, insertion, updating, and retrieval of data to build complete applications.
- Very high-level languages used to generate application code automatically.

SQL and QBE, mentioned earlier, are examples of 4GLs. The following briefly describes some other key types.

Forms generators. A forms generator is an interactive tool for rapidly designing data entry and display screens. It allows the user to define screen layout, specify what information should appear and where, and set visual properties such as color, boldness, underlining, and blinking. More capable forms generators support the creation of derived attributes using arithmetic operations or aggregates, and the specification of validation rules for data input.

Report generators. A report generator is a tool for producing formatted reports from database data. Like a query language, it allows the user to pose questions and retrieve data, but it provides far greater control over the appearance of the output. Some report generators automatically determine how output should be presented; others allow the user to design fully customized layouts using report-specific command languages. Two main varieties exist: language-oriented report generators, in which report definitions are expressed as commands in a sublanguage, and visually oriented report generators, in which a forms-like interface is used to define layout and content.

Graphics generators. A graphics generator retrieves data from the database and renders it as a chart or graph to reveal trends and relationships. Typical output types include bar charts, pie charts, line charts, and scatter plots.

Application generators. An application generator produces complete programs that interface with the database. It accelerates the development process by providing a library of prewritten modules — implemented in a high-level language — that encapsulate the fundamental functions most programs require. The user specifies what the program should accomplish; the application generator determines how to carry out the specified tasks.

2.3 Data Models and Conceptual Modeling

As noted earlier, a schema is written using a DDL — specifically, the DDL of a particular DBMS. This kind of language is, however, too low-level to convey the data requirements of an organization in a way that can be easily understood by the full range of stakeholders. What is needed is a higher-level descriptive framework: a data model.

Data model: An integrated collection of concepts for describing and manipulating data, the relationships between data, and the constraints on data in an organization.

A model is a representation of real-world objects, events, and their associations. It is an abstraction that focuses on the essential, inherent aspects of an organization while filtering out incidental detail. A data model represents the organization itself. It should provide the basic concepts and notation that enable database designers and end-users to communicate clearly and without ambiguity about the organization's data. A data model can be understood as consisting of three components:

1. A structural component — a set of rules that define how databases may be constructed.
2. A manipulative component — a definition of the types of operations permitted on the data, encompassing operations for updating, retrieving, and restructuring the database.
3. A set of integrity constraints — rules that ensure the data remains accurate.

The purpose of a data model is to represent data in a way that makes it understandable and, by extension, that facilitates database design. Aligned with the ANSI-SPARC architecture, three related data models can be identified:

1. An external data model, representing each user's view of the organization — sometimes called the Universe of Discourse (UoD).
2. A conceptual data model, representing the logical (or community) view in a manner that is independent of any specific DBMS.
3. An internal data model, expressing the conceptual schema in a form that the DBMS can understand and implement.

A wide variety of data models have been proposed in the literature. They fall into three broad categories: object-based, record-based, and physical. The first two describe data at the conceptual and external levels; the third describes data at the internal level.

2.3.1 Object-Based Data Models

Object-based data models describe data using concepts such as entities, attributes, and relationships. An entity is a distinct object in the organization — a person, place, thing, concept, or event — that is to be represented in the database. An attribute describes a specific aspect of the entity, and a relationship is an association between entities. The most commonly encountered types of object-based data model include:

- Entity-Relationship (ER)
 - Semantic
 - Functional
 - Object-oriented

The ER model has become one of the principal techniques for database design and forms the basis of the design methodology presented in this book. The object-oriented data model extends the concept of an entity to encompass not only attributes that describe the object's state but also the operations that define its behavior — encapsulating both state and behavior in a single construct.

2.3.2 Record-Based Data Models

In record-based models, the database is organized as a collection of fixed-format records, potentially of different types. Each record type defines a fixed number of fields, typically of fixed length. Three principal record-based logical data models exist: the relational data model, the network data model, and the hierarchical data model. The hierarchical and network models predate the relational model by roughly a decade, and their closer kinship with traditional file-processing concepts is correspondingly more apparent.

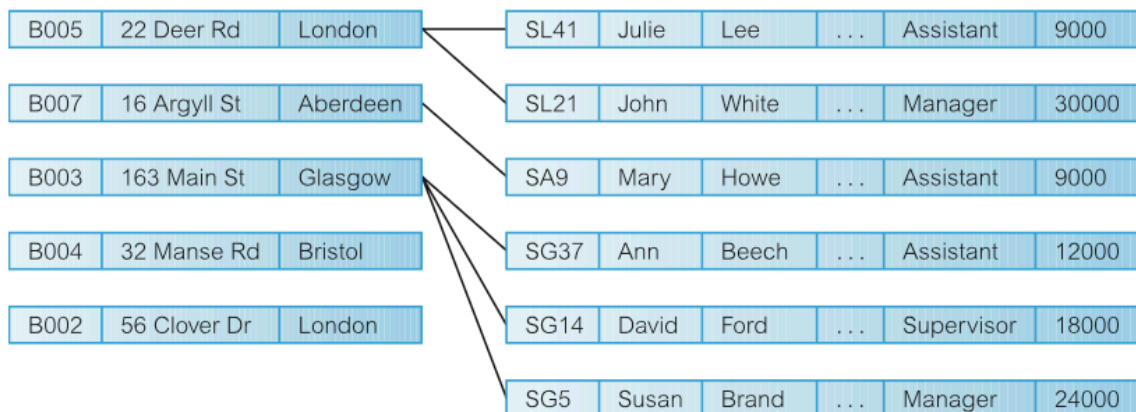
Relational data model. The relational data model is grounded in the mathematical concept of a relation. Data and relationships are both represented as tables, each with a set of uniquely named columns. Figure 2.4 shows a sample instance of a relational schema for part of the DreamHome case study, presenting branch and staff details. For example, it shows that employee John White is a manager earning £30,000 who works at branch B005 — which, from the first table, is located at 22 Deer Road in London. The relationship between Staff and Branch — that a branch office employs staff — is represented implicitly: the branchNo attribute in the Staff table corresponds to the branchNo in the Branch table, and it is this correspondence that establishes the relationship. There is no explicit link in the data structure itself. The relational model requires only that the database be perceived by users as tables; this applies to the logical structure of the database (the external and conceptual levels) but not to the physical storage structure, which may be implemented in a variety of ways.

Branch

branchNo	street	city	postCode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

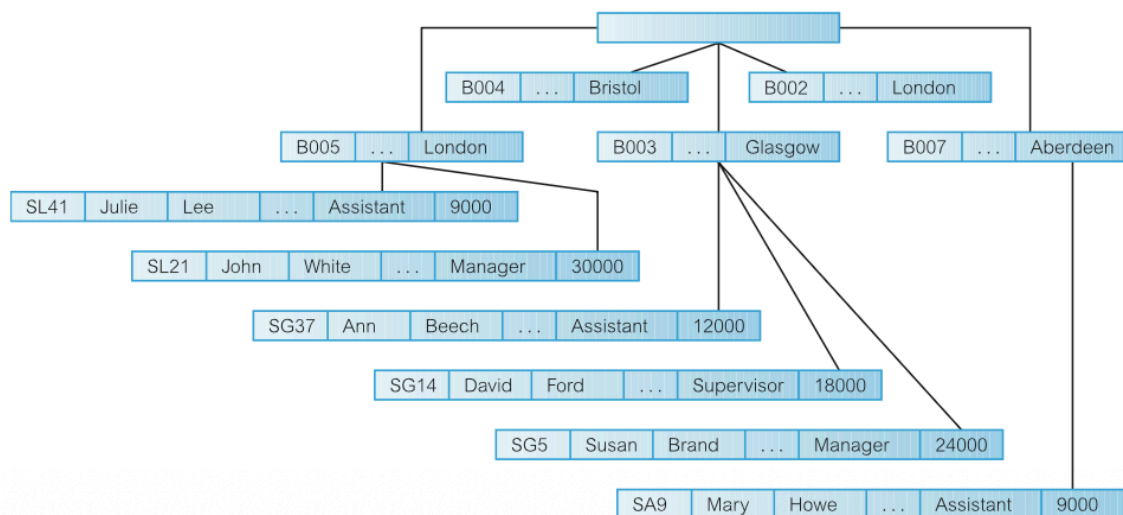
Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005



Network data model. In the network model, data is represented as collections of records and relationships are represented as sets. Unlike the relational model, where relationships are implicit, the network model makes relationships explicit through set structures that are implemented as pointers. Records are organized as generalized graph structures, with records as nodes (also called segments) and sets as edges. Figure 2.5 illustrates an instance of a network schema for the same data set shown in Figure 2.4. The most widely used network DBMS is Computer Associates' IDMS/R. The network data model is discussed further on the book's website.

Hierarchical data model. The hierarchical model is a constrained variant of the network model. Data is again represented as records and relationships as sets, but with a critical restriction: each node may have only one parent. The result is a tree structure, with records as nodes (segments) and sets as edges. Figure 2.6 illustrates an instance of a hierarchical schema for the same dataset. The predominant hierarchical DBMS is IBM's IMS, though IMS also provides non-hierarchical extensions. The hierarchical model is discussed further on the book's website.



Record-based logical data models are well suited to specifying the overall structure of a database and providing a higher-level description of its implementation. Their main weakness is that they offer limited facilities for explicitly expressing constraints on the data. Object-based data models, conversely, are richer in semantic content and more expressive for constraint specification, but lack the formal structural rigor of record-based models.

The vast majority of modern commercial database systems are based on the relational paradigm, whereas the earliest commercial systems used either the network or hierarchical model. Both of the older models require users to have knowledge of the physical organization of the data being accessed — they take a navigational approach, specifying how data is to be retrieved. Relational systems, by contrast, adopt a declarative approach, specifying only what data is to be retrieved and leaving the DBMS to determine how to obtain it. This declarative nature is a primary source of the relational model's data independence advantage.

2.3.3 Physical Data Models

Physical data models describe how data is actually stored in the computer, specifying record structures, record orderings, and access paths. Far fewer physical data models have been proposed compared with logical data models; the most notable are the unifying model and the frame memory model.

2.3.4 Conceptual Modeling

From an examination of the three-level architecture, it is clear that the conceptual schema is the core of the database. It supports every external view and is in turn supported by the internal schema — which is ultimately just the physical realization of the conceptual schema. The conceptual schema must be a complete and accurate representation of the enterprise's data requirements. If it is not, some information will be missing or incorrectly represented, and it will be impossible to fully implement one or more of the external views.

Conceptual modeling — or conceptual database design — is the process of constructing a model of an enterprise's information use that is entirely independent of implementation specifics: the target DBMS, application programming languages, hardware, or any other physical consideration. The product of this process is called a conceptual data model.

The term "logical model" is sometimes used in the literature as a synonym for conceptual model. In this book, however, we maintain a distinction between the two. A conceptual model is independent of all implementation details. A logical model, by contrast, assumes knowledge of the underlying data model of the target DBMS — for instance, the relational model.

2.4 Functions of a DBMS

This section examines the types of function and service that a DBMS should be expected to provide. Codd (1982) identified eight services that any full-scale DBMS should offer; we supplement these with two additional services that can reasonably be expected of a modern system.

(1) Data storage, retrieval, and update

A DBMS must furnish users with the ability to store, retrieve, and update data in the database.

This is the most fundamental function a DBMS performs. As the discussion in Section 2.1 makes clear, the DBMS must deliver this capability while completely shielding users from the internal physical implementation details — file organization, storage structures, indexing mechanisms, and so on.

(2) A user-accessible catalog

A DBMS must furnish a catalog in which descriptions of data items are stored and which is accessible to users.

A central feature of the ANSI-SPARC architecture is the recognition of an integrated system catalog that holds information about schemas, users, applications, and related metadata. The catalog must be accessible both to the DBMS internally and to end-users directly. A system catalog — also called a data dictionary — is a repository that stores "data about the data," or metadata. The volume and usage of this information varies from one DBMS to another, but a typical catalog holds:

- Names, types, and sizes of data items.
- Names of relationships.
- Integrity constraints on the data.
- Names of authorized users and the access rights granted to each — for example, insert, update, delete, or read access.
- External, conceptual, and internal schemas, and the mappings between them as described in Section 2.1.4.
- Usage statistics, such as transaction frequencies and counts of accesses to database objects.

The system catalog is one of the most critical components of the entire DBMS. Many of the software subsystems described in the next section depend on the catalog for the information they need to operate. Its benefits include:

- Data descriptions can be collected and stored in a single central location, supporting consistent management of data as an organizational resource.
- The meaning and purpose of data items can be formally defined, helping users understand the data they work with.
- Communication is simplified because precise definitions are stored and retrievable. The catalog may also identify the users who own or access particular data items.
- Redundancy and inconsistencies are easier to identify because all data descriptions are centralized.
- Changes to the database can be recorded for audit and tracking purposes.
- The impact of a proposed change can be assessed before it is applied, because the catalog records each data item, all its relationships, and all users who depend on it.
- Security enforcement is supported.
- Data integrity can be maintained.
- Audit trails can be generated.

Some authors draw a distinction between a system catalog and a data directory, the latter being specifically concerned with where and how data is physically stored. The ISO has adopted a standard for data dictionaries known as the Information Resource Dictionary System (IRDS) (ISO 1990, 1993). IRDS is a software tool for controlling and documenting an organization's information resources. It defines the tables that comprise the data dictionary and the operations available for accessing them. Throughout this book, the term "system catalog" is used to encompass all forms of repository information. Additional types of statistical information stored in the catalog to support query optimization are discussed in Section 23.4.1.

(3) Transaction support

A DBMS must furnish a mechanism that will ensure either that all the updates corresponding to a given transaction are made or that none of them is made.

A transaction is a sequence of actions carried out by a single user or application program that accesses or modifies the contents of the database. Simple examples in the DreamHome context include adding a new staff member, updating a salary, or removing a property from the register. A more complex example might involve deleting a staff member and simultaneously reassigning all the properties they managed to a different employee. In such a case, multiple changes to the database are required as a single logical unit. If the transaction fails partway through — perhaps due to a system crash — the database will be left in an inconsistent state: some changes will have been applied and others will not. The DBMS must be capable of rolling back the partial changes to restore the database to a consistent state before the transaction began.

(4) Concurrency control services

A DBMS must furnish a mechanism to ensure that the database is updated correctly when multiple users are updating the database concurrently.

Enabling multiple users to access shared data simultaneously is one of the primary motivations for using a DBMS. When all users are only reading data, concurrent access is relatively straightforward — readers cannot interfere with one another. However, when two or more users access the database at the same time and at least one is performing updates, interference can occur and produce inconsistencies.

Figure 2.7 illustrates the "lost update" problem. Two transactions, T1 and T2, execute concurrently. T1 is withdrawing £10 from an account with a current balance of £100; T2 is depositing £100 into the same account. If these transactions were executed serially, the final balance would correctly be £190 regardless of order. In the concurrent scenario, however, both T1 and T2 read the initial balance of £100. T2 adds £100, producing a balance of £200, and writes this back to the database. Meanwhile, T1 subtracts £10 from its own copy of the original balance, obtaining £90, and writes this value back — overwriting T2's update and effectively losing the £100 deposit.

Time	T ₁	T ₂	bal _x
t ₁		read(bal _x)	100
t ₂	read(bal _x)	bal _x = bal _x + 100	100
t ₃	bal _x = bal _x - 10	write(bal _x)	200
t ₄	write(bal _x)		90
t ₅			90

(5) Recovery services

A DBMS must furnish a mechanism for recovering the database in the event that the database is damaged in any way.

As noted in the discussion of transaction support, a failed transaction must be reversed so that the database can be returned to a consistent state. Failures may arise from a system crash, storage media failure, hardware or software errors that cause the DBMS to halt, or from a user deliberately aborting a transaction after detecting an error. In all such cases, the DBMS must provide mechanisms to detect the problem and restore the database to its last known consistent state.

(6) Authorization services

A DBMS must furnish a mechanism to ensure that only authorized users can access the database.

There are many scenarios in which certain data must be restricted from some users. Salary information, for instance, might appropriately be visible only to branch managers. More broadly, the database must be protected against unauthorized access — whether deliberate or accidental. The term "security" in the database context refers to this protection. The DBMS is expected to provide mechanisms that define and enforce security policies governing who may access the database and what operations they may perform on which data.

(7) Support for data communication

A DBMS must be capable of integrating with communication software.

Users typically access the database from workstations that may be co-located with the host machine running the DBMS or situated at remote locations connected via a network. In either case, requests reach the DBMS as communication messages and responses are returned in kind. A data communication manager (DCM) handles all such transmissions. Although the DCM is not itself part of the DBMS, a commercially viable DBMS must be capable of integrating with a variety of DCMs. Even single-site personal computer DBMSs should support operation over a local area network, enabling users to share a single centralized database rather than each maintaining a separate copy. This does not imply that the database must be physically distributed across the network — only that users at various locations can connect to and access a centralized database.

(8) Integrity services

A DBMS must furnish a means to ensure that both the data in the database and changes to the data follow certain rules.

Database integrity refers to the correctness and consistency of stored data. While related to security, integrity is a broader concern: it addresses the quality of the data itself rather than merely controlling who can access it. Integrity is expressed through constraints — consistency rules that the database must not be permitted to violate. For example, a constraint might specify that no staff member may manage more than 100 properties simultaneously. The DBMS should evaluate this constraint whenever a new property is assigned to a staff member and reject the assignment if the limit would be exceeded.

Beyond these eight core services, two additional capabilities can reasonably be expected of a modern DBMS:

(9) Services to promote data independence

A DBMS must include facilities to support the independence of programs from the actual structure of the database.

The concept of data independence was introduced in Section 2.1.5. In practice, data independence is achieved through a view or subschema mechanism. Physical data independence is the more straightforward of the two to achieve: many types of change to the physical characteristics of the database can be made without affecting any user views. Logical data independence is considerably harder to fully realize. The addition of new entities, attributes, or relationships can generally be accommodated without disrupting existing views, but their removal cannot. In some systems, any modification to an existing component of the logical structure is prohibited entirely.

(10) Utility services

A DBMS should provide a set of utility services.

Utility programs assist the DBA in administering the database efficiently. Some utilities operate at the external level and can therefore be developed by the DBA organization; others operate at the internal level and can only be provided by the DBMS vendor. Examples of vendor-supplied utilities include:

- Import facilities for loading the database from flat files, and export facilities for unloading data to flat files.
 - Monitoring facilities for tracking database usage and operational behavior.
 - Statistical analysis programs for examining performance metrics and usage patterns.
 - Index reorganization facilities for rebuilding indexes and managing overflow areas.
 - Garbage collection and space reallocation tools for physically removing deleted records from storage, consolidating the freed space, and redistributing it where needed.
-

Chapter Summary

- The ANSI-SPARC database architecture defines three levels of abstraction: external, conceptual, and internal. The external level consists of the various users' views of the database. The conceptual level is the community-wide view, specifying the complete information content of the database independent of storage considerations — encompassing all entities, their attributes, and their relationships, as well as data constraints and security and integrity information. The internal level is the computer's view of the database, specifying how data is physically represented, how records are ordered, what indexes and pointers exist, and so on.
- The external/conceptual mapping transforms requests and results between the external and conceptual levels. The conceptual/internal mapping transforms requests and results between the conceptual and internal levels.
- A database schema is a description of the database structure. Three types of schema correspond to the three ANSI-SPARC levels. Data independence makes each level immune to changes at lower levels. Logical data independence refers to the immunity of external schemas to changes in the conceptual schema. Physical data independence refers to the immunity of the conceptual schema to changes in the internal schema.
- A data sublanguage consists of two parts: a DDL for specifying the database schema and a DML for reading and updating the database. The data retrieval portion of a DML is called a query language.
- A data model is a collection of concepts for describing data, the operations that may be applied to it, and the integrity constraints it must satisfy. Data models fall into three broad categories: object-based, record-based, and physical. The first two are used at the conceptual and external levels; physical data models are used at the internal level.
- Object-based data models include the Entity–Relationship, semantic, functional, and object-oriented models. Record-based data models include the relational, network, and hierarchical models.
- Conceptual modeling is the process of constructing a detailed, implementation-independent representation of an organization's data requirements. The quality of the conceptual schema is central to the overall success of the database system; the time and effort invested in producing the best possible conceptual design are well spent.
- The functions and services expected of a full-scale multi-user DBMS include: data storage, retrieval, and update; a user-accessible catalog; transaction support; concurrency control and recovery services; authorization services; support for data communication; integrity services; services to promote data independence; and utility services.
- The system catalog is one of the fundamental components of a DBMS. It contains metadata — "data about the data" — and should be accessible to users. The Information Resource Dictionary System (IRDS) is an ISO standard that defines access methods for a data dictionary, enabling dictionaries to be shared and transferred between systems.

